

Practical 5 — Morphology & Add-Delete Table

Objective: To study how words are formed from smaller meaning-bearing units (morphemes) and represent transformations using Add-Delete tables.

Introduction

Morphology is the study of word structure. Words are built from morphemes — the smallest units that carry meaning. This practical introduces the Add-Delete table approach to systematically record how derived words are formed from base words.

Types of Morphemes

- **Root morpheme:** Base form carrying core meaning (e.g., `play`).
- **Prefix:** Added to the beginning (e.g., `un-` in `unhappy`).
- **Suffix:** Added to the end (e.g., `-ing` in `playing`).
- **Infix/affix changes:** Some transformations involve spelling changes (doubling letters, replacing letters).

Add-Delete Table — Concept

An Add-Delete table records: the base word, the derived word, what part (if any) was deleted from the base, and what part was added to create the derived word. Example columns: `Base Word` | `Derived Word` | `Delete` | `Add` | `Description`.

Example (manual)

Base Word	Derived Word	Delete	Add	Description
play	playing		ing	Suffix addition
happy	unhappy		un	Prefix addition
run	running		ning	Doubling + suffix
study	studied	y	ied	Replace + suffix
write	writer	e	er	Delete + suffix

```
In [2]: # Utilities: compute a plausible Add-Delete decomposition between two words
import pandas as pd
import numpy as np
from typing import Tuple

def compute_add_delete(base: str, derived: str) -> Tuple[str, str, str]:
    """Return (delete_part, add_part, description) describing a contiguous change f
    if base == derived:
        return ('', '', 'No change')
```

```

minlen = min(len(base), len(derived))
# find leftmost index where they differ
left = 0
while left < minlen and base[left] == derived[left]:
    left += 1
# find common suffix length without overlapping the left index
r = 0
while (r < (len(base) - left)) and (r < (len(derived) - left)) and base[-1 - r] == derived[-1 - r]:
    r += 1
del_part = base[left: len(base) - r] if left <= len(base) - r else ''
add_part = derived[left: len(derived) - r] if left <= len(derived) - r else ''
# simple description heuristics
if del_part == '' and add_part == '':
    desc = 'No change'
elif del_part == '':
    desc = 'Addition (suffix/prefix or insertion)'
elif add_part == '':
    desc = 'Deletion'
else:
    # small heuristics for common patterns
    if del_part == 'y' and add_part == 'ied':
        desc = 'Replace (y -> ied) + suffix'
    elif len(del_part) == 1 and add_part.endswith('er'):
        desc = 'Replace + suffix'
    else:
        desc = 'Replace/Delete + Addition'
return (del_part, add_part, desc)

```

```

In [3]: # Demonstrate auto-generated Add-Delete table for sample words
examples = [('play', 'playing'), ('happy', 'unhappy'), ('run', 'running'), ('study', 'studied')]
rows = []
for b,d in examples:
    del_p, add_p, desc = compute_add_delete(b,d)
    rows.append((b, d, del_p if del_p!='' else '-', add_p if add_p!='' else '-', desc))
auto_df = pd.DataFrame(rows, columns=['Base Word', 'Derived Word', 'Delete', 'Add', 'Description'])
print('Auto-generated Add-Delete table:')
auto_df

```

Auto-generated Add-Delete table:

```

Out[3]:

```

	Base Word	Derived Word	Delete	Add	Description
0	play	playing	—	ing	Addition (suffix/prefix or insertion)
1	happy	unhappy	—	un	Addition (suffix/prefix or insertion)
2	run	running	—	ning	Addition (suffix/prefix or insertion)
3	study	studied	y	ied	Replace (y -> ied) + suffix
4	write	writer	—	r	Addition (suffix/prefix or insertion)
5	create	creation	e	ion	Replace/Delete + Addition
6	be	being	—	ing	Addition (suffix/prefix or insertion)

Manual Example Table (matches common morphological analyses)

Below is a curated Add-Delete table that records commonly taught morphological analyses for a few examples. Use this as a reference when comparing with the automatic outputs.

```
In [4]: manual_examples = [
    ('play', 'playing', '-', 'ing', 'Suffix addition'),
    ('happy', 'unhappy', '-', 'un', 'Prefix addition'),
    ('run', 'running', '-', 'ning', 'Doubling + suffix'),
    ('study', 'studied', 'y', 'ied', 'Replace + suffix'),
    ('write', 'writer', 'e', 'er', 'Delete + suffix'),
]
man_df = pd.DataFrame(manual_examples, columns=['Base Word', 'Derived Word', 'Delete', 'Add', 'Description'])
man_df
```

Out[4]:

	Base Word	Derived Word	Delete	Add	Description
0	play	playing	—	ing	Suffix addition
1	happy	unhappy	—	un	Prefix addition
2	run	running	—	ning	Doubling + suffix
3	study	studied	y	ied	Replace + suffix
4	write	writer	e	er	Delete + suffix

Procedure — How to build an Add-Delete table for a pair of words

1. Identify the root/base word.
2. Align base and derived from the left to find the first differing character.
3. Align from the right to find the longest unchanged suffix (without overlapping the left difference).
4. The substring removed from the base is the **Delete** column; the substring added in the derived is the **Add** column.
5. Note spelling changes (e.g., doubling, vowel changes) in the Description column.

Exercises

1. For each pair below, compute the Add-Delete decomposition and justify the Description column:
 - inform -> informal
 - hope -> hoping

- stop -> stopped
- beauty -> beautiful
- decide -> decision

2. Extend the automatic function to handle multi-step changes (e.g., prefix + suffix together).

Conclusion

The Add-Delete table is a compact way to record morphological transformations. Building simple utilities helps in linguistic annotation and in understanding algorithms like stemming and lemmatization used in NLP.